Tartalomjegyzék

1. Bevezetés **2**
1.1. Célkitűzés . . . 2
1.2. A mesterséges intelligencia és gépi tanulás . . . 2
1.3. Kaotikus neurális hálózatok . . . 8
1.4. Gépi tanulás alapú időjárás-előrejelző modellek . . . 9
1.5. MLWP-modellek kaotikus tulajdonságai . . . 12

2. Adat, módszer **14**
2.1. A trajektóriamodell . . . 15
2.2. Kaotikus viselkedés számszerűsítése . . . 15

1. Bevezetés

1.1. Célkitűzés

A huszadik század eleje óta – többek között a „bergeni iskola” neves kutatóinak köszönhetően – ismerjük a légkör működését leíró nemlineáris egyenletrendszert, az úgynevezett kormányzó egyenleteket. Az időjárás-előrejelzés hagyományosan ezen egyenletrendszer integrálásán alapul, melyet szuperszámítógépeken futó numerikus modellek hajtanak végre (*Numerical Weather Prediction, NWP*). Az utóbbi években jelentős áttörés történt az időjárás-előrejelzés területén: a 2020-as évek elején megjelentek a gépi tanulás alapú időjárás-előrejelző modellek (*Machine Learning-based Weather Prediction, MLWP*). A gépi tanulás alapú modellek rekonstruált historikus időjárási adatokból – úgynevezett reanalízis adatokból –, illetve operatív NWP-előrejelzésekből tanulják meg a légköri folyamatok mintázatait, és ezáltal képesek a kiinduló adatok – a peremfeltételek – ismeretében predikciót adni a jövő várható időjárására. Az eltérő megközelítés folytán több előny és hátrány is megjelenik a hagyományos NWP-modellekhez képest.

Az MLWP-modellek legnagyobb előnye, hogy rendkívül rövid idő alatt képesek előrejelzést készíteni, melyek pontossága összevethető a hagyományos modellekével. Az első ilyen jelentősebb modell, az NVIDIA FourCastNet 2022-es publikációjában szereplő adatok is ezt támasztják alá: a modell 2 másodperc alatt képes egyhetes időjárás-előrejelzést készíteni, pontossága pedig az egyik legjobb NWP-modell, a European Centre for Medium-Range Weather Forecasting (*ECMWF*) IFS nevű modelljével összemérhető [26]. A 2022 óta eltelt években sorra jelentek meg az MLWP-modellek (pl. Huawei Pangu-Weather, 2023; Google GraphCast, 2023; FuXi, 2023; FengWu, 2023; ECMWF-AIFS, 2024; illetve néhány ensemble modell: Google GenCast, 2024; ECMWF AIFS ENS, 2025), és a tapasztalatok egybehangzóak: az előrejelzések rendkívül gyorsak és pontosak, ami megfelelő indok arra, hogy a modelleket még tovább fejlesszék.

A további fejlesztés mellett szól az adatalapú megközelítésből adódó alapvető hátulütők kiküszöbölésére való törekvés is. Egyik ilyen abból ered, hogy a modellek gyakran valamilyen négyzetes hiba, mint a négyzetes középhiba (*Mean Squared Error, MSE*) alapú költségfüggvény minimalizálására vannak betanítva. Ennek következtében igyekeznek elkerülni a szélsőséges értékek predikcióját, mivel tévesen megállapított kiugró érték jelentős büntetést eredményezne. Az ilyen módon előálló szélsőérték-kerülést nevezzük „smoothing”-nak, utalva arra, hogy a kiugró csúcsok mintegy elmosásra kerülnek.

Másrészt – mivel a modell nem ismeri a légköri kormányzó egyenleteket, hanem csak korábbi időjárási adatok mintázataiból tanul – elvileg lehetséges a fizikai valósággal inkonzisztens előrejelzés készítése is. Ilyen volt pl. az ECMWF-AIFS egy korábbi verziójának az a hibája, hogy negatív csapadékmennyiséget jelzett előre. A modellfejlesztők a valósághű predikciókat elérendő fizikai kényszerfeltételek bevezetésével korlátozzák a modellt [23].

Többek között a fentebb említett hibák miatt a szakirodalom jelenleg nem tekinti az MLWP modelleket a légkör megfelelő szimulátorainak [6]. A szakirodalomban azonban még nem sok kutatás tűzte ki céljául a gépi tanulás alapú időjárás-előrejelző modellek kaotikus tulajdonságainak vizsgálatát. Kutatásomban azt szeretném megvizsgálni, hogy a fizikai törvények helyett adatokon alapuló megközelítés kimutathatatlanná teszi-e az előrejelzésekben a légkör egy alapvető tulajdonságát, a káoszt. Amennyiben a modellek nem tudják reprodukálni a kaotikus jegyeket, úgy hamis biztosságot adhatnak a hosszú távú előrejelzések megbízhatóságára [30]. Ha azonban észlelhető a káosz a modell előrejelzéseiben, az azt jelenti, hogy az adatalapú tanítás következtében mégsem vész el minden fizikai törvényszerűség.

1.2. A mesterséges intelligencia és gépi tanulás

A mesterséges intelligencia fogalmát John McCarthy, Marvin Minsky, Claude Shannon és Nathaniel Rochester alkották meg 1956-ban a Dartmouth workshop-on (*Dartmouth Summer Research Project*

on Artificial Intelligence). Felvetett elvük azon alapult, hogy „a tanulás minden aspektusa, vagy az intelligencia bármely más jellemzője elvileg olyan pontosan leírható, hogy egy géppel azt szimulálni lehet" [24, 8]. A tanulás általános fogalma új ismeretek, képességek és viselkedésminták elsajátítását vagy módosítását jelenti. A mintázatok megismerése által lehetővé válik a jövőre vonatkozó előrejelzések készítése. Gépek esetében a tanuláshoz szükséges tapasztalatszerzés adatok alapján, mintázatfelismerés által történik; ezt nevezzük gépi tanulásnak [2].

Az első gépi tanulás alapú technológia az 1990-es években lett népszerű, amikor megjelentek az elektronikus levelezőrendszerekben a spamfilterek: a szűrő felhasználói bejelentésekből származó adatokon lett betanítva, megtanulva, hogy milyen jellemzői vannak egy spam e-mailnek. Az ezt követő évtizedekben egyre több gépi tanulás alapú technológia jelent meg, gondolhatunk itt a mindennapokban gyakran használt beszédfelismerő eszközökre, vagy a közösségi média tartalomajánló algoritmusaira [13]. A gépi tanulást 1959-ben Arthur Samuel amerikai informatikus így definiálta [20]:

> *A gépi tanulás képessé teszi a számítógépeket a tanulásra anélkül, hogy explicit módon programoznánk őket.*

Egzaktabb definíciót nyújt Tom Mitchell szintén amerikai informatikus megfogalmazása [22]:

> *Egy számítógépes program valamilyen T feladat elvégzése érdekében – melynek sikerességét valamely P teljesítményt értékelő mérőszám jellemzi – E tapasztalatból tanul, ha a T elvégzése során nyújtott teljesítménye P mérőszámmal mérve az E tapasztalat megszerzésével javul.*

A gépi tanulás olyan problémák megoldásában lehet rendkívül hasznos, amik túl komplexek, vagy nem ismert algoritmikus megoldásuk. Ha megnézzük, hogy a gép milyen mintázatokat ismert fel vagy kapcsolatokat tanult meg, akkor az ember is új, addig még fel nem fedezett korrelációkat vehet észre, ami által a megértésünk javulhat, ezt nevezzük adatbányászatnak (*data mining*). A példák, tapasztalatok, amik segítségével a gép tanul, a tanító adathalmaz (*training set*) részét képezik (*sample*) [13].

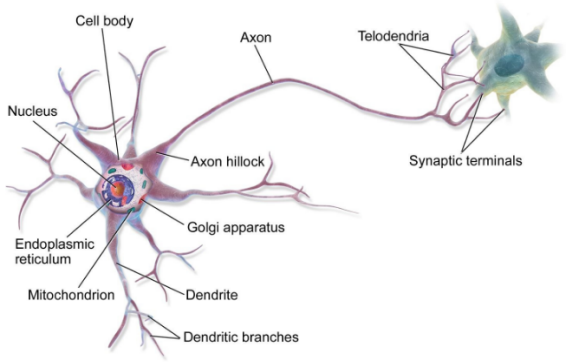
A gépi tanulásos rendszereket csoportosíthatjuk aszerint, hogy humán felügyelet alatt van-e tanítva (felügyelt – *supervised*, felügyelet nélküli – *unsupervised*, félig felügyelt – *semisupervised*, megerősítéses – *reinforcement* tanulás); menet közben tud-e fokozatosan tanulni (*online* és *batch* tanulás); illetve hogy az új adatokat a már ismertekhez hasonlítja (példa alapú – *instance-based* tanulás) vagy a tanító adathalmazban mintázatokat ismer fel, és ez alapján előrejelzést tud adni (modell alapú – *model-based* tanulás). A legfontosabb fogalmak a fentiek közül a felügyelt, felügyelet nélküli, illetve megerősítéses tanulás [13].

A felügyelt tanulás során a tanító adathalmaz tartalmazza a kívánt kimenetet / helyes megoldást (ezt nevezzük címkének – *label*). Példa lehet erre, hogy szeretnénk betanítani egy modellt, hogy korábbi eladási adatokat ismerve jelezze előre, mennyit ér egy eladásra szánt autó. Az autónak a tulajdonságai (*feature*; mint a kora, megtett km, márka stb.) adottak. Az előrejelezni kívánt numerikus mennyiséget hívjuk *predictor*-nek vagy *target*-nek (jelen esetben ez az autó ára). Az előrejelzéshez használt algoritmus többféle lehet, legismertebb példái a lineáris regresszió és a neurális hálózat architektúrák. Felügyelet nélküli tanulás során a tanító adathalmaz nincsen felcímkézve, a modell maga tanulja meg a jellemzőket elkülöníteni. Legismertebb példa a klaszterezés (*clustering*), amikor a modell a vizsgált adatokat a megfigyelt tulajdonságaik alapján osztályozza. Megerősítéses tanulás esetén a tanuló rendszer (az ágens – *agent*) megfigyeli a környezetét és a kialakított irányelvének (*policy*) megfelelően tetteket hajt végre. A tetteiért jutalmat (*reward*) vagy büntetést (*penalty*) kap. Az ágens célja a jutalom maximalizálása, ami érdekében iteratívan frissíti az irányelvét [13].

A gépi tanulás alapú modellek problémái legtöbbször a tanító adatból származnak: gondot jelent a nem megfelelő mennyiségű, minőségű, nem elég reprezentatív, vagy irreleváns adat. Ha a modell túl komplex a feladathoz, felléphet a túlillesztés (*overfitting*), ami azt jelenti, hogy az illesztett függvény

helytelenül a zajt is figyelembe vette. Ha a modell túl egyszerű, azt alulillesztésnek (*underfitting*) nevezzük. Az elkészült gépi tanulás alapú modelleket ezen hibák kiszűrése érdekében validálják, melynek népszerű eszköze az igazságmátrix (*confusion matrix*) [13].

A gépi tanulás hagyományosan statisztikai módszerek segítségével történik a megfigyelt adatok alapján. A gépi tanulás múlt században megalkotott, és az utóbbi évtizedekben elterjedt technikája egészen más megközelítést kínál; ezek a mesterséges neurális hálózatok (*Artificial Neural Network, ANN*). Az ANN-ek segítségével az adatbázisban fennálló komplex kapcsolatok is felismerhetőek [31]. Az ANN-eket 1943-ban mutatta be Warren McCulloch amerikai neurofiziológus és Walter Pitts számítógépes idegtudós. Az ANN-t mesterséges neuronok alkotják, melyek működését a biológiai neuronok ihlették, melyet az 1.1. ábra mutat be.



1.1. ábra. A biológiai neuron szerkezete. Az ábra Géron *Hands-On Machine Learning with Scikit-Learn and TensorFlow* (2017) [13] könyvéből származik.

A biológiai neuron (idegsejt) az agykéregben előforduló sejt, melynek főbb részei a sejttest (*cell body*), a dendritek (*dendrite*), illetve a nagyon hosszú nyúlványa, az axon. Az axon telodendronokra bomlik, melyek csúcsán kis szinapszisok találhatóak, melyek a dendritekbe tudnak kapcsolódni és a neuronok kis elektromos impulzusokkal történő kommunikációjáért felelősek. Ha a neuron adott idő alatt elég jelet kap, maga is aktiválódik és jeleket ad le. A biológiai neuronok közti kapcsolódási pont a szinapszis, ami erősödik, ha a két neuron gyakran aktiválódik együtt [8]. Az agyban a neuronok egymás utáni rétegekbe vannak rendeződve, így lesz képes az emberi agy komplex feladatok elvégzésére is [13].

Az ANN rétegebe rendeződött neuronokból áll. A rétegekben csomópontok vannak, az információ feldolgozásáért felelős mesterséges neuronok pedig kapcsolatokon keresztül kommunikálnak egymással [32]. A csomópontok a biológiai neuron sejttestének feleltethetőek meg, a kapcsolatok pedig hasonlóan működnek, mint az axonok és dendritek. A mesterséges neuronok közti kapcsolat erősségét a súlyok (*weight*) határozzák meg [8].

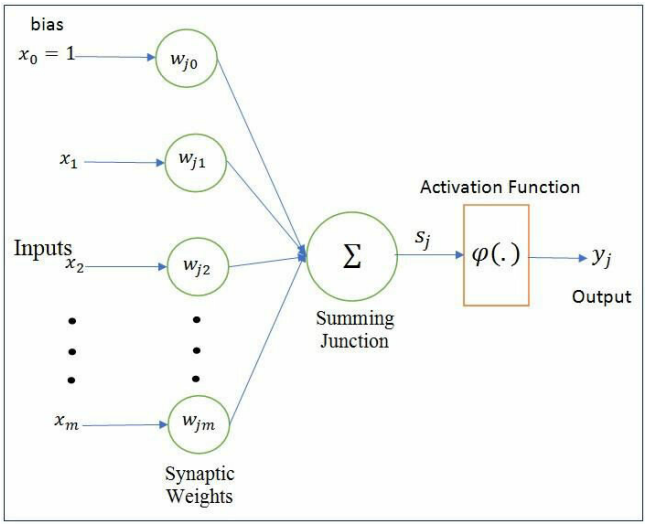
A McCulloch és Pitts által megalkotott ANN kijelentéslogikán alapult. A neuronok egy vagy több bináris (*on/off* típusú) bemenetet (*input*) fogadtak, és egyetlen kimenetet (*output*) adtak. A biológiai neuronok aktiválódását úgy imitálták, hogy a mesterséges neuron csak akkor adott kimenetet, ha a kapott bemenetek száma elért egy bizonyos értéket.

Az egyik legegyszerűbb neurális hálózat a Perceptron, melyet 1957-ben Frank Rosenblatt mutatott be. Úgynevezett küszöb logikai egységek (*Treshold Logic Unit, TLU*) a mesterséges neuronok, az input és output értékek pedig numerikusak; minden input – fontosságának megfelelően – súlyozva van. A súlyozott bemenetek mellett egy b konstans értéket felvevő elfogultság (*bias*) is hozzáadásra szokott kerülni. A TLU egy súlyozott összeget számol, majd egy nemlineáris aktivációs függvényt hattat az eredményre, és ezt adja outputnak. Az aktivációs függvényre azért van szükség, hogy a lineárisnál bonyolultabb kapcsolatokat is képes legyen megtanulni az ANN. Aktivációs függvény lehet pl. a Heaviside-féle egységfüggvény, ReLu (*REctified Linear Unit*, ami a negatív értékeket lenullázza), de

gyakran valamilyen szigmoid függvényt alkalmaznak a célra, mint a tangens hiperbolikusz vagy a logisztikus függvény. A mesterséges neuron működését az alábbi egyenlet foglalja össze:

$$s_j = \sum_{i=1}^{m} w_{ij} \cdot x_i + b_j; \quad \hat{y_j} = \varphi(s_j),$$

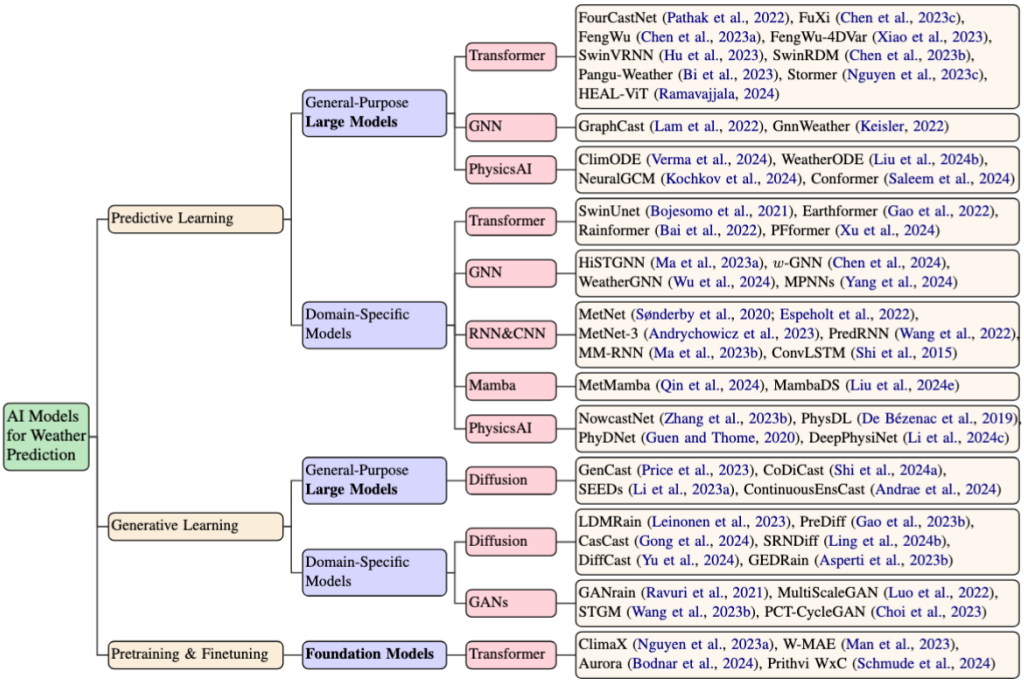
ahol x_i az input, b_j a bias, w_{ij} a súlymátrix, s_j a súlyozott összeg, φ az aktivációs függvény, $\hat{y_j}$ pedig az output. A fentebb leírt mesterséges neuron működését mutatja be a 1.2. ábra. [32, 13, 17].



1.2. ábra. A mesterséges neuron sematikus felépítése. Az ábra Sharkawy (2020) [32] cikkéből származik.

Egyszeres neuronréteg azonban csak egyszerűbb feladatok elvégzésére alkalmazható; ha bonyolultabb problémákat szeretnénk megoldani, több rétegre van szükség. A kettőnél több rétegből álló ANN-t mély neurális hálózatnak (*Deep Neural Network, DNN*) nevezzük. Ennek legegyszerűbb példája a többrétegű Perceptron (*Multilayer Perceptron, MLP*), ami több rétegnyi TLU-ból áll, melyeket rejtett rétegeknek (*hidden layers*) nevezünk. Minden réteg teljesen hozzá van kapcsolva a szomszédos réteghez (*Fully Connected, FL*) [13].

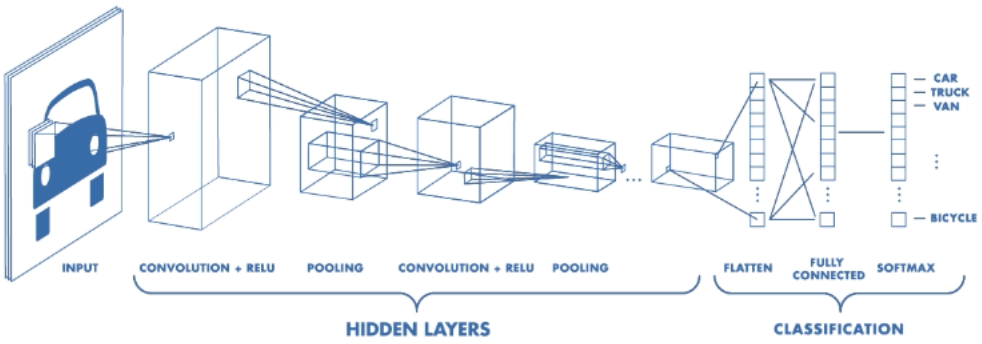
A bonyolultabb feladatok elvégzésére tehát mélytanulásos (*Deep Learning, DL*) technológiák alkalmasak. Mélytanulásos architektúra többek között a konvolúciós neurális hálózat (*Convolutional Neural Network, CNN*), rekurrens neurális hálózat (*Recurrent Neural Network, RNN*), a gráfalapú neurális hálózat (*Graph Neural Network, GNN*), a transzformer architektúra (*Transformer*), vagy a Vision Transformer [33]. Az időjárás-előrejelzésben használt MLWP-modellek áttekintését mutatja a 1.3. ábra.



1.3. ábra. Az MLWP-modellek áttekintése. Az ábra Shi et al. (2025) [33] cikkéből származik.

Ahogy az ábrán is látható, az MLWP-modelleknek alapvetően három fajtája van. Legelterjedtebbek az általános célú nagy modellek (*General-Purpose Large Models*), melyek többféle időjárási paraméter globális előrejelzésére alkalmasak. Az operatív időjárás-előrejelzésben kevésbé elterjedtebbek a domén specifikus modellek (*Domain-Specific Models*, melyekkel regionális előrejelzést lehet készíteni egyetlen változóra), illetve az alapmodellek (*Foundation Models*, melyekkel még további finomhangolás, specifikus feladathoz adaptálás lehetséges). Az is megfigyelhető, hogy az utóbbi években a CNN és RNN alapú időjárás-előrejelző modellek háttérbe szorultak a GNN és transzformer architektúrájú modellek mellett.

A CNN architektúrát LeCun et al. mutatta be 1995-ben [19]. Az architektúra sturkturált rácsadatok, mint pl. a képek feldolgozására alkalmas; klasszikusan a gépi látás területén (pl. képosztályozás) CNN modelleket használnak [33]. A CNN-nek négy rétege van (konvolúciós réteg – *convolutional layer*, gyűjtőréteg – *pooling layer*, teljesen összekötött réteg – *fully connected layer* és a nemlineáris réteg – *nonlinearity layer*), melyeket a 1.4. ábra szemléltet. A rétegek szerepét egy képosztályozási feladaton keresztül mutatom be.



1.4. ábra. A CNN szerkezete. Az ábra Purwono et al. (2022) [27] cikkéből származik.

A CNN az inputként kapott képet egy mátrixként látja, ahol a mátrix minden eleme az adott pixel

fényintenzitását reprezentálja. A konvolúciós réteg (*convolutional layer*) a képekből a jellemzőket (*feature*, pl. élek, textúrák, minták) nyeri ki. A gyűjtőréteg (*pooling layer*) konvolúciós rétegeket kapcsol össze; szerepe a paraméter- és ezáltal számítási teher csökkentése a konvolúciós réteg kimeneteinek alulmintavételezett reprezentáció készítésével. A teljesen összekapcsolt réteg (*fully connected layer*, más néven konvolúciós output réteg – *convolutional output layer*) bemenete az utolsó pooling réteg. Az adatot a következő rétegbe továbbítás előtt vektorrá lapítja. Az utolsó réteg a nemlineáris réteg vagy aktivációs függvény (*nonlinearity layer / activation function*); ez a réteg adja meg a neurális hálózat végső kimenetét, a válaszát az adott feladatra [27].

Az RNN-eket 2001-ben mutatta be Medsker & Jain [21], mint időfüggő vagy szekvenciális adatokkal kapcsolatos feladatok (pl. idősor-előrejelzés) végzésére alkalmas architektúrákat. A modell rekurrens módon tanul a kapott adatsorozatból egy belső rejtett állapot segítségével, ami tartalmazza a korábbi rejtett állapotokat és az aktuális bemenetet. A tanulás folyamatát az alábbi egyenletek írják le:

$$h_t = \sigma(\mathbf{W}_x x_t + \mathbf{W}_h h_{t-1} + b_h); \quad y_t = \sigma(\mathbf{W}_y h_t + b_y),$$

ahol a t-edik időpillanatban h_t a rejtett állapot, x_t az input, y_t az output; $\mathbf{W}_x, \mathbf{W}_h, \mathbf{W}_y$ súlymátrixok, b_h, b_y bias-ok, σ pedig az aktivációs függvény. A CNN-nél és RNN-nél sokkal fontosabb architektúrák azonban az időjárás-előrejelzésben a GNN-ek és a transzformer alapú neurális hálózatok [33].

A GNN-t 2008-ban mutatták be Scarselli et al. [29]. Az architektúra $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ $\mathcal{V}$ csúccsal és $\mathcal{E}$ éllel rendelkező gráf szerkezetű adatok feldolgozására alkalmas, ahol a csúcsok a vizsgált dolgokat, az élek pedig a köztük fennálló kapcsolatokat reprezentálják. Ha a GNN a tér- és időbeli függőségeket is jól modellezik, tér-idő GNN-ekről (*Spatio-temporal GNN, ST-GNN*) beszélünk, melyeket a $\mathcal{G}_t = (\mathcal{V}, \mathcal{E}, t)$ jelöléssel szoktak illetni. Itt a csúcsok és az élek térbeli kapcsolatokat írnak le; a vizsgált dolog i térbeli helyen és t időpillanatban felvett vektorát a v_t^i vektor reprezentálja. A térbeli függőségek figyelembevétele az egyes csomópontok szomszédaikkal való kommunikációján (*message passing*) keresztül valósul meg. Az időbeli kapcsolatokat szekvenciális modellekkel (mint az RNN) lehet figyelembe venni. A csomópontok h_t^i rejtett állapotai a v_{t+1}^i üzenetvektorokkal kerülnek frissítésre az alábbi módon:

$$v_{t+1}^i = \sum_{j \in N(i)} M_t(h_t^i, h_t^j, e_{ij}); \quad h_{t+1}^i = \sigma(h_t^i, v_{t+1}^i),$$

ahol $N(i)$ az i-edik csomópont szomszédjai, M_t az üzenetküldő függvény, e_{ij} pedig az i-edik és j-edik csomópont közti élet jellemzi [33].

Az utóbbi évek legnépszerűbb mesterséges intelligencia-alapú modelljeinek alapjai a transzformer architektúrák. 2017-ben jelent meg Vaswani et al. a transzformer neurális hálózatokat bemutató azóta már nagyon híressé vált cikke, az „Attention Is All You Need” [35]. Az architektúra a figyelem mechanizmus (*attention mechanism*) alapján működik; szekvenciális lépések közbeiktatása nélkül képes az adatban fennálló kapcsolatokat megragadni. A figyelem mechanizmus az alábbiak szerint írható le:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V},$$

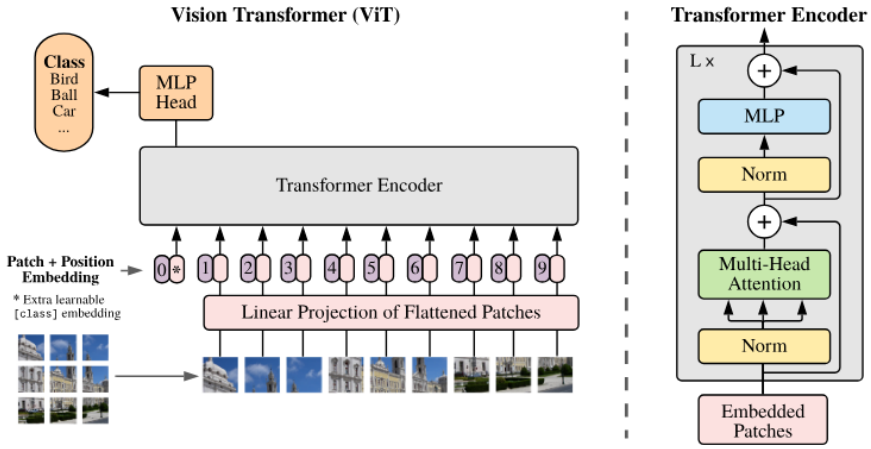
ahol d_k a $\mathbf{K}$ kulcsmátrix (*key matrix*) dimenziója, $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$ a lekérdezési mátrix (*query matrix*), $\mathbf{K} \in \mathbb{R}^{m \times d_k}$ a kulcsmátrix, $\mathbf{V} \in \mathbb{R}^{m \times d_v}$ az értékmátrix (*value matrix*). Ezeket a mátrixokat az $\mathbf{X} \in \mathbb{R}^{n \times d}$ bemeneti szekvenciális adatokat tartalmazó mátrix alapján az alábbiak szerint számolják ki:

$$\mathbf{Q} = \mathbf{X}\mathbf{W_q}, \quad \mathbf{K} = \mathbf{X}\mathbf{W_k}, \quad \mathbf{V} = \mathbf{X}\mathbf{W_v},$$

ahol $\mathbf{W_q} \in \mathbb{R}^{d \times d_k}$, $\mathbf{W_k} \in \mathbb{R}^{d \times d_k}$, $\mathbf{W_v} \in \mathbb{R}^{d \times d_v}$ rendre a **Q, K, V** mátrixokhoz tartozó tanulható súlymátrixok [33].

A Vision Transformer-t (*ViT*) Dosovitskiy et al. mutatta be 2021-ben [11]. Ez egy olyan transzformer, amely képek feldolgozására lett kifejlesztve. A ViT a bemenetként kapott képet rácsos szerkezetben részekre (*patch*) osztja; ezeket az adat-darabokat (úgynevezett *tokeneket*) alakítja aztán vektorrá

és dolgozza fel. Az így előkészített tokeneket továbbítják aztán egy transzformerbe, ahol a figyelem mechanizmus már alkalmazható. A Vision Transformer szerkezetét mutatja a 1.5. ábra.



1.5. ábra. A ViT szerkezete. Az ábra Dosovitskiy et al. (2021) [11] cikkéből származik.

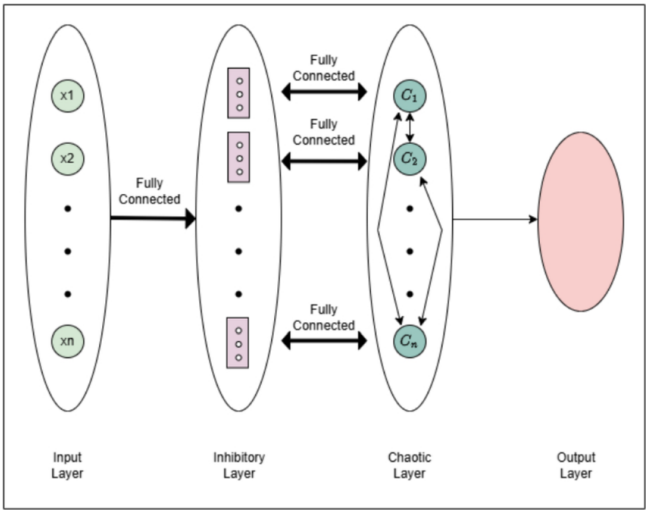
1.3. Kaotikus neurális hálózatok

A káosz görög eredetű szó, eredeti jelentése „tátongó űr". Máshogy megfogalmazva a káosz egy komplex rendszer viselkedésében megnyilvánuló teljes zűrzavar vagy kiszámíthatatlanság. A káoszt a kezdeti feltételekre való rendkívüli érzékenységgel szokás azonosítani, ami a hosszútávú idősor-előrejelzést lehetetleníti el. Ezt először Henri Poincaré francia matematikus figyelte meg a 19. században, majd 1963-ban Edward Lorenz angol matematikus és meteorológus fogalmazta meg a káoszelméletet; tőle származik a „pillangóhatás" kifejezés is. Lorenz egy modellt készített, ami a légköri mozgásokat hivatott reprezentálni. A számításai során látszólag elhanyagolható kerekítéseket végzett, azonban észrevette, hogy ez az előrejelzésben rendkívül nagy eltéréseket okozott. Ez a kezdeti feltételekre való érzékenység koncepciója. Az 1989-ben Benoît Mandlebrot fedezte fel a fraktálok (önhasonló objektumok; a „káosz mintázatai") jelentőségét a kaotikus attraktorok leírásában. [5, 3].

Ahhoz, hogy egy rendszer kaotikusnak nevezhessünk, az alábbi jellemzőknek kell teljesülnie:

- Nem jelenik meg periodikusság.
- Kezdeti feltételekre való érzékenység.
- Kaotikus mozgást nehéz vagy lehetetlen előrejelezni.
- Véletlenszerűnek látszó (de valójában determinisztikus) mozgás.
- Nemlineáris.

A kaotikus neurális hálózatokat (*Chaotic Neural Network, cNN*) – a káoszelmélet és neurális hálózatok ötvözetét – Aihara et al. mutatták be 1990-ben [1]. A cNN-ek a kaotikus rendszerek komplex mintázatait képesek reprezentálni és változatos megoldásokat tudnak felfedezni. A cNN neuronjai nemlineáris aktivációs függvények által kaotikus dinamikát mutatnak, amely egy ún. bifurkációs paraméter változtatásával szabályozható. Ez azért hasznos, mert így a neurális hálózat komplex – kaotikus – kapcsolatokat is felfedezhet az adatok közt. [3]. A kaotikus neuron szerkezetét a 1.6. ábra mutatja be.



1.6. ábra. A kaotikus neuron szerkezete Crook & Scheper (2001) [9] alapján. Az ábra Anusree & Pramod (2025) [3] cikkéből származik.

A bemutatott kaotikus neuron három rétegből áll. A bemeneti (*input*) réteg fogadja a bejövő dinamikus jeleket. A réteg teljesen össze van kötve (*fully connected*) egy gátló neuronokból álló klaszterek alkotta réteggel (*inhibitory layer*). Egy-egy ilyen klasztert alkotó neuronok mindegyik a következő, kaotikus (*chaotic*) réteg ugyanazon neuronjához kapcsolódik. A kaotikus réteget alkotó minden egység teljes összeköttetésben van a többi egységgel [3].

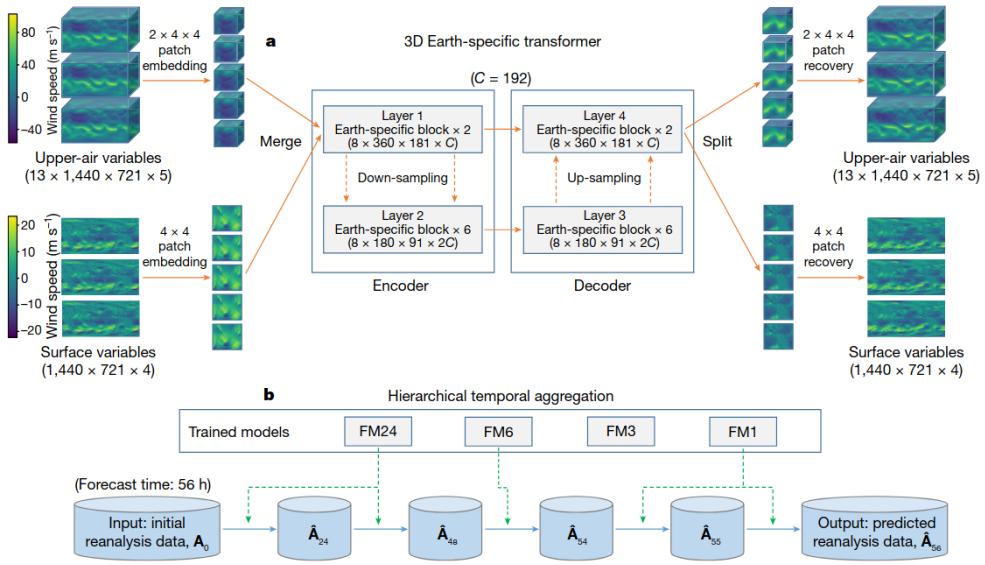
A cNN-ek tehát le tudják képezni a kaotikus jellemzőket. Jelenleg azonban főként arra használják a technológiát, hogy a tanítás során a modell ne ragadjon bele lokális minimumokba [3]. Az Echo State Network-öt (*ESN*) – egy olyan neurális hálózat, ami bizonyos körülmények között képes kaotikus tulajdonságokat produkálni – Ortiz et al. (2025) [25] már használták a meteorológia területén, felhőzet-előrejelzésre. Azonban a kaotikus neurális hálózatok gépi tanulás alapú időjárás-előrejelző modellekbe való integrálása még nem terjedt el széleskörűen, nem találkoztam ilyen jellegű tanulmányokkal a szakirodalmi kutatásom során.

1.4. Gépi tanulás alapú időjárás-előrejelző modellek

Az időjárás-előrejelzés területén a gépi tanulást már több, mint 25 éve használják (ld. pl. Hung et al., 2009, akik ANN-t használtak az eső előrejelzésére Bangkokban [16]), azonban csak az utóbbi években ért el a technológia olyan fejlettséget, hogy operatív, több időjárási változóra is kiterjedő előrejelzést lehessen készíteni segítségével [10].

Az első jelentősebb ANN architektúrán alapuló MLWP-modell az NVIDIA 2022-ben bemutatott, Fourier neurális hálózat alapú FourCastNet (*FOURier ForeCASTing Neural NETwork*) nevű modellje. Az architektúra alapötlete a fentebb már ismertetett ViT. Azonban a ViT erőforrásigénye a pixelszámokkal négyzetesen nő, így sokdimenziós bemeneti adatoknál már nem érdemes használni. Ezen probléma áthidalására fejlesztették ki Guibas et al. 2022-ben az Adaptív Fourier Neurális Operátorokat (*Adaptive Fourier Neural Operator, AFNO*). A módszer a függvények Fourier-terén hat, ami sok számítást leegyszerűsít (pl. a konvolúcióból szorzás lesz). Az AFNO lényegében a ViT módszert váltja fel [26, 12].

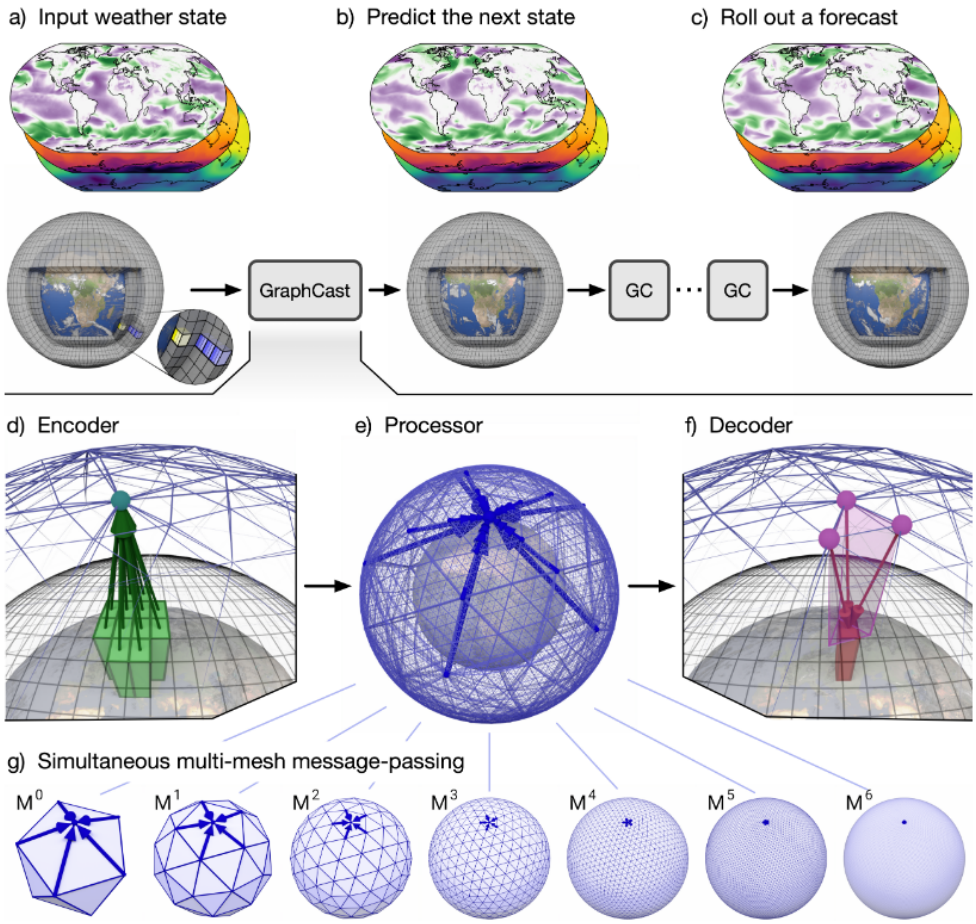
Szintén híres MLWP-modell a Huawei által 2023-ban bemutatott Pangu-Weather. A modell egy ún. 3D-s Föld-specifikus transzformer (*3D Earth-Specific Transformer, 3DEST*), melynek szerkezetét a 1.7. ábra mutatja be.



1.7. ábra. A 3DEST szerkezete. Az ábra Bi et al. (2023) [4] cikkéből származik.

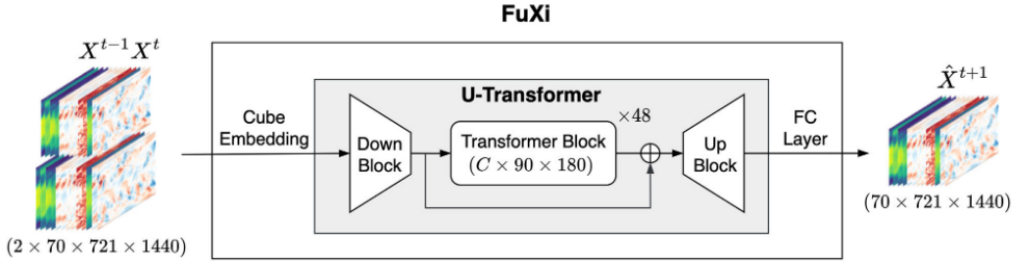
A 3DEST architektúra a bemenő adatokon ún. *patch embedding*et végez, ami során a térbeli felbontást lecsökkenti és egy 3D-s kockába sűríti. Ezt követően a hálózat a 3D-s adatokat egy enkóder-dekóder architektúrába továbbítja, ami jelen esetben egy Swin-transzformer, ami a ViT egy fajtája. Az enkóder-dekóder architektúra által átalakított adatokon ezt követően *patch recovery*t hajt végre, hogy az eredeti felbontást visszakapjuk. A modell célja középtávú (7+ nap) előrejelzések készítése volt, ami eléréséhez egy ún. *hierarchical temporal aggregation* módszert, egy „mohó algoritmust” (*greedy algorithm*) alkalmaztak. Mivel a neurális hálózatok rövid időtávra (1, 3, 6, 24 óra) pontosabban tudnak előrejelezni, a hosszabb távú előrejelzéseket rövidebb távúakból hierarchikusan állítják elő. Például ha 56 órára készítenek előrejelzést, először két 24 órás előrejelzést készítenek. A fennmaradó $56 - 2 \cdot 24 = 8$ órát egy 6 órás és 2 db 1 órás előrejelzéssel fedik le. Ezáltal az előrejelzési hibák kevésbé halmozódnak fel, mintha pl. 56 db 1 órás előrejelzést raknának össze [4].

A következő bemutatott modell a Google által szintén 2023-ban publikált, GNN-alapú GraphCast modellje. A GNN-ek működése fentebb már említésre került. A modell a számításokat egy ún. *multi-mesh*-en – magas felbontású, ikozaéderes gráfon – végzi. A modellben enkóder-processzor-dekóder architektúra található. Az 1 GNN-rétegből álló enkóder 6-szoros iteratív finomítás után az ikozaéder csomópontjait (*node*-jait) a földgömbre vetíti; ezzel létrejönnek $M^0 ... M^6$ rácshálózat, ahol M^0 a legdurvább, M^6 pedig a legfinomabb felbontást jelenti. A teljes *multi-mesh* mind a 7 rácshálózatot magában foglalja. A 16 GNN-réteget magában foglaló processzor a *multi-mesh*-en az információ továbbítását tanulja. A szintén 1 GNN-rétegből álló dekóder a *multi-mesh*-en megtanult jellemzőket visszavetíti a földrajzi fokhálózatra. A *multi-mesh* egyik előnye, hogy – szemben a hagyományos földrajzi fokhálózattal – térben homogén, nincsen pl. a pólusokon *node* felgyülemlés. A másik legnagyobb előnye, hogy a többféle felbontású rácshálózatokat magában foglaló *multi-mesh*-en hatékonyabb az információ továbbítása: egy távoli pontba pl. először a legdurvább M^0-n, majd egyre finomabb rácshálókon halad az információ. A GraphCast modell működését a 1.8. ábra szemlélteti [18].



1.8. ábra. A GraphCast modell működése. Az ábra Lam et al. (2023) [18] cikkéből származik.

A Chen et al. által 2023-ban bemutatott FuXi MLWP-modell három részből áll: a kocka beágyazásból (*cube embedding*), egy U-transzformerből, illetve egy egy teljesen összekötött (*fully connected*) rétegből. A *cube embedding* célja a bemenő sokdimenziós adat idő- és térbeli dimenzióinak, ezáltal redundanciájának csökkentése. Az adatok feldolgozását egy U-transzformer végzi. Az adatdimenzió csökkentését egy alulmintavételező egység, a *Down Block* végzi. Ezt követően 48 egymás utáni Swin-transzformer típusú ViT-ből dolgozza fel az adatokat. Végül az adatok dimenziójának növelését egy felülmintavételező egység, az *Up Block* végzi. A modell felépítését a 1.9. ábra mutaja be [7].



1.9. ábra. A FuXi modell áttekintő szerkezete. Az ábra Chen et al. (2023) [7] cikkéből származik.

Az ECMWF által 2024-ben bemutatott Artificial Intelligence Forecasting System (AIFS) modell több korábban bemutatott MLWP-modell előnyeit ötvözi; egyszerre GNN és transzformer alapú is. Az ERA5 rácshálózatról a processzor alacsonyabb felbontású oktaéderes gráfhálózatára vetítő enkóder és a visszalakítást végző dekóder GNN-transzformer („*GNN attention*”) architektúrával rendelkezik. A processzor tisztán transzformer alapú, 16 db Swin-transzformer rétegből áll. A processzor a rács-

hálózatot alkotó gráf minden pontjának környezetét is figyelembe veszi egy adott levágási (*cut-off*) sugáron belül.

1.5. MLWP-modellek kaotikus tulajdonságai

Azt, hogy megállapítsuk, hogy egy modell tudja-e a kaotikus jegyeket produkálni, legegyszerűbben az előrejelzésben valamilyen kezdeti perturbáció növekedésének vizsgálatával tehetjük meg. Ha a hiba exponenciálisan nő (azaz találunk pozitív Ljapunov-exponenst), a káosz jelen van a predikcióban.

Ezt vizsgálták 2023-ban Selz & Craig kis perturbációval ellátott ensemble (együttes) előrejelzések segítségével. A vizsgált modellek az MLWP-típusú Pangu-Weather és az NWP-típusú ICON modellek voltak; a modell-előrejelzések perturbációra adott válaszát vizsgálták. Azért fontos, hogy a perturbáció kicsi legyen, mert a nagy perturbáció elnyomja a modell-specifikus effektusokat, és az MLWP-modell úgy viselkedik, mint a hagyományos NWP-modellek. Az NWP-modellek alapján várható lenne, hogy a légkör kaotikussága miatt a hozzáadott kis hiba exponenciálisan kezd el nőni, ami az időjárás-előrejelzésnek egy inherens időbeli felső korlátot ad („pillangó-hatás"). Ezzel szemben Selz & Craig azt tapasztalták, hogy a Pangu-Weather nem képes reprodukálni ezt a kezdeti exponenciális hibanövekedést, amivel egy hamis biztonságérzetet ad a hosszútávú időjárás-előrejelzések megbízhatóságára [30].

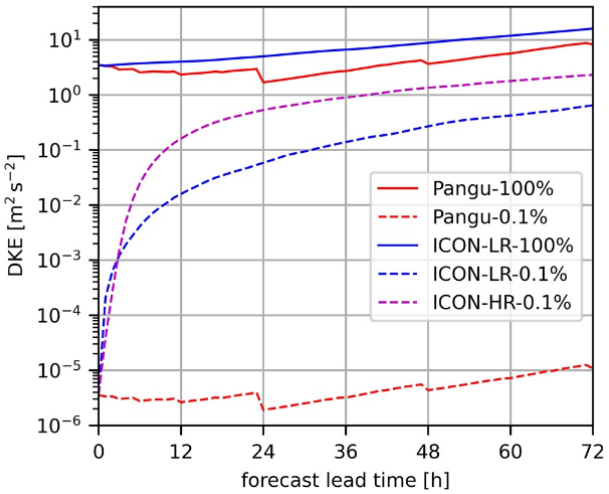
Vizsgálatuk során kis- és nagy felbontású rácshálózatot is alkalmaztak. A nagy felbontású rácson (2,5 km) a konvekció – a légkör egyik legkevésbé lineáris folyamata – is megfigyelhető legyen. A vizsgálat esettanulmány-jellegű, 2021. június 26. 00 UTC inicializációs idejű előrejelzéseket használtak fel. Az időpont-választás oka, hogy ekkor Észak-Amerika felett – nyár lévén – nagyon jelentős konvekciós folyamatok zajlottak; azonban eredményeiket az egész Földre átlagolva adták meg [30].

A hibanövekedés számszerűsítésére egy ilyen célra gyakran használt mennyiséget, a 300 hPa-on számolt kinetikus energia-különbséget (*Difference Kinetic Energy, DKE*) alkalmazták:

$$\mathrm{DKE} = \mathrm{Var}(u) + \mathrm{Var}(v),$$

ahol u és v a horizontális szélkomponensek [30].

A kutatók kis- és nagyobb perturbációkat is végeztek; eredményeiket a 1.10. ábra foglalja össze.



1.10. ábra. Az ICON és Pangu-Weather (*Pangu*) nagy (*100%*) és kicsi (*0.1 %*) perturbációkra adott válaszai a DKE növekedésén keresztül bemutatva. A *HR* a nagy felbontást (2.5 km), a *LR* az alacsony felbontást (20 km) jelent. Az ábra Selz & Craig (2023) [30] cikkéből származik.

A nagy perturbáció során alátámasztásra került a várakozásuk, hogy az elnyomja az MLWP-modell jellegzetességeit: az erősen perturbált Pangu-Weather (*Pangu-100%*) hasonlóan viselkedett, mint az

erősen perturbált ICON (*ICON-LR-100%*). A kicsit perturbált ICON (*ICON-0.1%*) esetében is az NWP-modellektől elvárható eredményre jutottak: a hiba nagyon gyorsan kezdett nőni, míg szaturációba nem került (48 óra után a hibanövekedési ütem kb. megegyezett az erősen perturbált ICON-éval). Ha az alacsonyabb felbontású, kicsit perturbált modellt (*ICON-LR-0.1%*) a magasabb felbontásúhoz (*ICON-HR-0.1%*) hasonlítjuk, megfigyelhető, hogy az alacsonyabb felbontású alulbecsli a hibát már néhány óra elteltével. Ennek oka az lehet, hogy a kis felbontás miatt nincsenek megfelelően reprezentálva a hibához jelentősen hozzájáruló nemlineáris konvektív folyamatok [30].

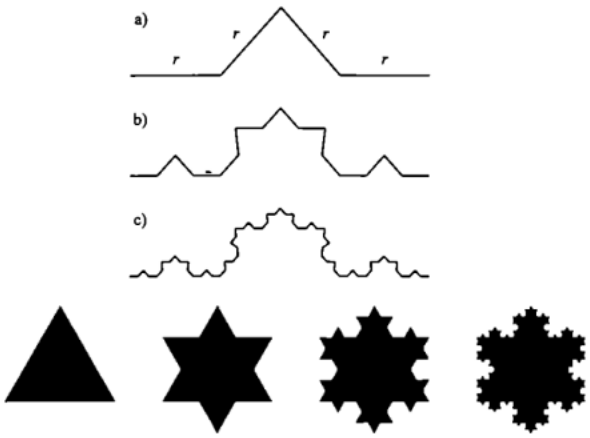
A kicsit perturbált Pangu-Weather modell hibanövekedésének időbeli lefutása nagyon hasonló az erősebben perturbálthoz; az eltolás a hibában a perturbáció mértékével arányos. A Pangu-Weather az ICON modellhez képest alulbecsli a hibát és konstans hibanövekedést mutat az exponenciális helyett, ami – helytelenül – sokkal hosszabb távú előrejelezhetőséget sugall [30].

Azonban a fent említett vizsgálat elvégzéséhez – tekintve, hogy az ensemble előrejelzések sok tagból állnak – rendkívül magas számítási kapacitásra van szükség. Selz & Craig pl. az ECMWF Atos nevű szuperszámítógépét használták. A Pangu-Weather kísérletek 16, az alacsony felbontású ICON-kísérletek 2900, a magas felbontású ICON-kísérletek pedig 1300000 számítási órába kerültek egyenként [30].

Ennél olcsóbb módszerek lehetnek az előrejelzési idősorban való kaotikus viselkedés észlelése, vagy trajektóriamodellezés.

Az előrejelzési idősorban való kaotikus viselkedés észlelésére felhasználható módszerekkel foglalkozik Pánis et al. 2020-as cikke [28].

A kaotikus rendszereket fázistérben (pl. sebesség-kitérés koordináta-rendszer) ábrázolva komplex, de rendezett, fraktálszerű alakzatot kapunk. A fraktálok olyan geometriai alakzatok, melyek kerülete és felülete nincs definiálva és önhasonlóak; ezt szemlélteti a 1.11. ábrán feltüntetett híres példa, a Koch-görbe, melynek területe véges marad ugyan, de kerülete a végtelenhez tart [34].



1.11. ábra. A Koch-görbe szerkesztése és fraktálszerkezete. Az ábrák Tél & Gruiz Kaotikus dinamika című tankönyvéből (2002) származnak [34].

A fraktálokat fraktáldimenzióval szokás jellemezni, ami síkbeli (térbeli) alakzatoknál egynél (kettőnél) nagyobb, de kettőnél (háromnál) kisebb tört vagy irracionális szám; a dimenzió nagysága az alakzat tagoltságával arányos. Fedjük le ezeket az alakzatokat ε karakterisztikus méretű dobozokkal (*box*-okkal). Míg pl. egy hagyományos síkbeli alakzat lefedéséhez ε^{-2}-el arányos az alakzat egyre finomabb lefedéséhez szükséges dobozok száma, addig fraktáloknál a kitevő egy nem egész szám, a fraktáldimenzió lesz:

$$N(\varepsilon) \propto \varepsilon^{-D_0},$$

ahol $\varepsilon \to 0$, N az alakzat lefedéséhez szükséges minimális számú doboz, D_0 pedig a fraktáldimenzió.

Átrendezve:

$$D_0 = \lim_{\varepsilon \to 0} \frac{\ln N(\varepsilon)}{\ln 1/\varepsilon}$$

adja meg a fraktáldimenziót. A box-counting módszer arra épül, hogyha tudunk ilyen fraktáldimenziót találni, akkor a vizsgált idősor kaotikus jeleket mutat [34, 28].

Másik módszer az idősor-adatokban a káosz észlelésére a korrelációs dimenzió (D_2) vizsgálata. A korrelációs dimenzió kiszámítása során megszámoljuk, hogy hány pontpár ($C(\varepsilon)$) van egymáshoz közelebb, mint egy nagyon kicsi ε küszöbérték:

$$D_2 = \lim_{\varepsilon \to 0} \frac{\ln C(\varepsilon)}{\ln \varepsilon}.$$

Ha találunk korrelációs dimenziót, az idősor kaotikus [28].

Következő módszer a Ljapunov-exponens keresése. Ha a kaotikus attraktor egy **r** pontja körül két közeli pontból indítunk részecskéket, távolságuk a lépések n számának növelésével exponenciálisan nőni fog:

$$\Delta r_n = \Delta r_0 e^{\bar{\lambda} n},$$

ahol Δr_0 a kezdeti (kicsi) távolságuk, $\bar{\lambda}$ pedig a fázistérhez rendelhető átlagos Ljapunov-exponens. Kaotikus rendszerekben $\bar{\lambda} > 0$. Ha az idősorban sikerül pozitív Ljapunov-exponenst találni – pl. egyenesillesztéssel –, akkor a rendszer, ahonnan az idősor származik, kaotikus [28].

A gépi tanulást a kaotikus viselkedés detektálására is használhatjuk. Alkotható olyan modell, amely a kaotikus minták felismerésére van tanítva, így az ember számára nehezen felismerhető vagy észrevétlenül maradt mintázatokat is képes felismerni [28].

A kaotikus jelek detektálására használhatóak lagrange-i trajektória modellek is: szimulálható segítségükkel, hogy a légkör egy térfogatdarabja (pl. egy szennyezés) hogyan változtatja térfogatát, növeli időben exponenciálisan a szálainak a hosszát, majd áll rá a kaotikus attraktorra [14].

A terjedési modellek célja, hogy a meteorológiai viszonyok (szélvektor) ismeretében modellezzék, hogy a légkörben egy részecske hogyan terjed. A terjedési modellek időjárás-előrejelző modellekkel vannak meghajtva, azok szélviszony-előrejelzései szolgáltatják a széladatokat a részecske mozgásának modellezéséhez. A terjedési modellekkel figyelembe lehet venni a felszínközeli viszonyokat, mint a súrlódás, azonban az bonyolulttá teszi a modellt. Ha alacsonyabb nyomású izobárszinteken vizsgáljuk a részecskék terjedését, dominánsan csak a szél fogja befolyásolni a részecske terjedését, így a súrlódás elhanyagolhatóvá válik.

A szimulált trajektóriák felhasználásával is ki lehet mutatni a káosz jelenlétét, legegyszerűbben a topologikus entrópia (h) segítségével tehető ez meg. A mennyiség jellemzi egy kezdetben L_0 hosszú fonaldarab megnyúlását:

$$L(t) = L_0 \cdot e^{ht} \quad (t \gg 1/h),$$

tehát a mozgás bonyolultságával van kapcsolatban: minél nagyobb h, annál bonyolultabb lesz a mozgás (jobban megnyúlik a kezdeti fonaldarab). Ha $h > 0$, a mozgás kaotikus. A topologikus entrópia becslése sok részecske pályájának nyomon követésével lehetséges [15].

2. Adat, módszer

Vizsgálataim során 2 dimenziós trajektóriamodellt hajtottam meg 2026. június 1-én UTC szerint 0 órakor készült ECMWF-IFS, illetve ECMWF-AIFS szél-előrejelzésekkel. Az előrejelzések középtávúak (360 órára előre készülnek el 6 órás lépésekkel), determinisztikusak, és jelentős számú izobárszintre (1000, 925, 850, 700, 600, 500, 400, 300, 250, 200, 150, 100, 50 hPa) elkészülnek $0,25° \times 0,25°$ felbontással.

2.1. A trajektóriamodell

A trajektóriamodell Pythonban íródott.

A modell bemeneteként beállítható, hogy melyik előrejelzési .grib fájlt kívánjuk felhasználni, milyen tértartományból és milyen sűrűséggel indítjuk a részecskéket, melyik izobárszinten vizsgálódunk, mennyi időre futtatjuk a szimulációt (a maximum az előrejelzések által meghatározott 360 óra), illetve hogy milyen időbeli sűrűséggel szeretnénk elmenteni a részecskék helyzetét.

Mivel a modellelőrejelzések csak 6 óránként állnak rendelkezésre, a kód lineáris interpolációt végez a köztes időpillanatok szélsebességének kiszámítására, amivel egy időben folytonosan változó szélsebesség-mezőt hoz létre.

Az elengedett részecskék trajektóriájának kiszámításához a következő differenciálegyenlet-rendszert szükséges megoldani minden részecskére:

$$\frac{d\lambda}{dt} = u \cdot C \cdot \frac{1}{\cos\varphi},$$

$$\frac{d\varphi}{dt} = v \cdot C,$$

ahol u a keleti (zonális)-, v pedig az északi (meridionális) szélkomponens m/s-ban; C egy váltószám, ami a m-t °-ba váltja át ($C = \frac{180°}{\pi \cdot R}$, ahol $R = 6371000\ m$ a Föld sugara); λ a részecske (keleti) hosszúsági koordinátája, φ az (északi) szélességi koordinátája; t pedig a szimulációs idő másodpercben. A 0-val való osztást úgy akadályozzuk meg, hogy a szélességi köröket csak $-89,9°$ és $89,9°$ között vesszük. A differenciálegyenlet-rendszert az elterjedt Runge-Kutta 4(5) numerikusan integráló módszer segítségével oldjuk meg. A Runge-Kutta módszer adaptív időlépésekkel dolgozik. Az esetlegesen fellépő túl nagy időlépés alkalmazását úgy akadályoztuk meg, hogy a maximális lépésközt 10 percre állítottuk. A relatív és abszolút hibatoleranciát is 10^{-3}-ra állítottuk.

2.2. Kaotikus viselkedés számszerűsítése

A modellezett trajektóriák felhasználásával topologikus entrópiát számoltam. Ehhez a szimuláció minden időlépésében szükséges kiszámolni a részecskéket képzeletben összekötő, egyre bonyolultabb alakot felvevő fonal hosszát. Ez úgy tehető meg, hogy az adott időlépésben a részecskék koordinátáinak ismeretében kiszámolom, milyen messze vannak egymástól a szomszédos részecskék, és ezeket a távolságokat összeadom. Ha a részecskék elég sűrűen vannak elhelyezkedve, ezzel a módszerrel viszonylag pontosan megkapható a fonal megnyúlásának időfüggése, a kapott adatokra való illesztéssel pedig egyszerűen meghatározható a h topologikus entrópia.

A rendelkezésre álló adatok $0,25° \times 0,25°$ felbontásúak, ami kb. 27 km × 27 km-es felbontást jelent. Ez trajektóriák modellezésekor meglehetősen durva felbontásnak számít, a szakirodalomban ennél jóval finomabb felbontást – pl. 300-600 m – használnak [15]. A hiba áthidalására a részecskéket egymástól $0,005°$-ra indítottam, ami kb. 500 m-t jelent. A modell minden lépésben térbeli lineáris interpolációval határozza meg a szélsebességet a részecskék pillanatnyi pozíciójában.

A fonal hosszát szemilogaritmikus diagramon ábrázoljuk az idő függvényében, így egy egyenes illesztésével, az egyenes meredekségeként lehet meghatározni a topologikus entrópiát.

Hivatkozások

[1] Aihara, K. et al. Chaotic neural networks. *Physics Letters A*, 144(6):333–340, March 1990. https://www.sciencedirect.com/science/article/pii/037596019090136C.

[2] Alzubi, J. et al. Machine Learning from Theory to Algorithms: An Overview. *Journal of Physics: Conference Series*, 1142(1):012012, November 2018. https://doi.org/10.1088/1742-6596/1142/1/012012.

[3] Anusree, M. et al. Understanding chaotic neural networks: A comprehensive review. *Nonlinear Dynamics*, 113(19):25389–25404, October 2025. https://doi.org/10.1007/s11071-025-11503-7.

[4] Bi, K. et al. Accurate medium-range global weather forecasting with 3D neural networks. *Nature*, 619(7970):533–538, July 2023. https://www.nature.com/articles/s41586-023-06185-3.

[5] Biswas, H. R. et al. Chaos Theory and Its Applications in Our Real Life. 2018.

[6] Bonavita, M. On Some Limitations of Current Machine Learning Weather Prediction Models. *Geophysical Research Letters*, 51(12):e2023GL107377, 2024. _eprint: https://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1029/2023GL107377, https://onlinelibrary.wiley.com/doi/abs/10.1029/2023GL107377.

[7] Chen, L. et al. FuXi: a cascade machine learning forecasting system for 15-day global weather forecast. *npj Climate and Atmospheric Science*, 6(1):190, November 2023. https://www.nature.com/articles/s41612-023-00512-1.

[8] Choi, R. Y. et al. Introduction to Machine Learning, Neural Networks, and Deep Learning. *Translational Vision Science & Technology*, 9(2):14, February 2020. https://doi.org/10.1167/tvst.9.2.14.

[9] Crook, N. et al. A Novel Chaotic Neural Network Architecture. In *ESANN'2001 proceedings - European Symposium on Artificial Neural Networks*, pages 295–300, Bruges, Belgium, 2001. D-Facto public.

[10] de Burgh-Day, C. O. et al. Machine learning for numerical weather and climate modelling: a review. *Geoscientific Model Development*, 16(22):6433–6477, November 2023. https://gmd.copernicus.org/articles/16/6433/2023/.

[11] Dosovitskiy, A. et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale, June 2021. arXiv:2010.11929 [cs.CV], http://arxiv.org/abs/2010.11929.

[12] Guibas, J. et al. Adaptive Fourier Neural Operators: Efficient Token Mixers for Transformers, March 2022. arXiv:2111.13587 [cs.CV], http://arxiv.org/abs/2111.13587.

[13] Géron, A. *Hands-On Machine Learning with Scikit-Learn and TensorFlow*. O'Reilly Media, Inc., Sebastopol, California, 2017.

[14] Haszpra, T. *A RePLaT modell és alkalmazása légköri szennyeződések terjedésének vizsgálatára*. PhD thesis, Eötvös Loránd Tudományegyetem, 2014. http://hdl.handle.net/10831/9274.

[15] Haszpra, T. et al. Ensemble-based analysis of the pollutant spreading intensity induced by climate change. *Scientific Reports*, 9(1):3896, March 2019. https://www.nature.com/articles/s41598-019-40451-7.

[16] Hung, N. Q. et al. An artificial neural network model for rainfall forecasting in Bangkok, Thailand. *Hydrology and Earth System Sciences*, 13(8):1413–1425, August 2009. https://hess.copernicus.org/articles/13/1413/2009/hess-13-1413-2009.html.

[17] Kelleher, J. D. *Deep Learning*. The MIT Press, Cambridge, Massachusetts, September 2019.

[18] Lam, R. et al. GraphCast: Learning skillful medium-range global weather forecasting, August 2023. arXiv:2212.12794 [cs.LG], http://arxiv.org/abs/2212.12794.

[19] Lecun, Y. et al. Convolutional networks for images, speech, and time series. 1995.

[20] Lee, A. et al. Machine Learning Has Arrived! *Ophthalmology*, 124(12):1726–1728, December 2017. https://linkinghub.elsevier.com/retrieve/pii/S0161642017315634.

[21] Medsker, L. R. et al. Recurrent Neural Networks. 2001.

[22] Mitchell, T. M. The Discipline of Machine Learning. 2006.

[23] Moldovan, G. et al. An update to ECMWF's machine-learned weather forecast model AIFS, September 2025. arXiv:2509.18994 [physics], http://arxiv.org/abs/2509.18994.

[24] Moor, J. The Dartmouth College Artificial Intelligence Conference: The Next Fifty Years. *AI Magazine*, 27(4):87–87, December 2006. https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/1911.

[25] Ortiz, E. L. et al. Exploring low-resource weather forecasting with echo state network-based architectures and satellite data. *Knowledge-Based Systems*, 323:113692, July 2025. https://www.sciencedirect.com/science/article/pii/S0950705125007385.

[26] Pathak, J. et al. FourCastNet: A Global Data-driven High-resolution Weather Model using Adaptive Fourier Neural Operators, February 2022. arXiv:2202.11214 [physics], http://arxiv.org/abs/2202.11214.

[27] Purwono, P. et al. Understanding of Convolutional Neural Network (CNN): A Review. *International Journal of Robotics and Control Systems*, 2(4):739–748, 2022. https://pubs2.ascee.org/index.php/IJRCS/article/view/888.

[28] Pánis, R. et al. Detection of chaotic behavior in time series, December 2020. arXiv:2012.06671 [physics.comp-ph], http://arxiv.org/abs/2012.06671.

[29] Scarselli, F. et al. The Graph Neural Network Model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2008. http://ieeexplore.ieee.org/document/4700287/.

[30] Selz, T. et al. Can Artificial Intelligence-Based Weather Prediction Models Simulate the Butterfly Effect? *Geophysical Research Letters*, 50(20):e2023GL105747, 2023. _eprint: https://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1029/2023GL105747, https://onlinelibrary.wiley.com/doi/abs/10.1029/2023GL105747.

[31] Sharifani, K. et al. Machine Learning and Deep Learning: A Review of Methods and Applications. 10(07), 2023. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4458723.

[32] Sharkawy, A.-N. Principle of Neural Network and Its Main Types: Review. *Journal of Advances in Applied & Computational Mathematics*, 7:8–19, August 2020. https://univ-brest.hal.science/hal-03114305.

[33] Shi, J. et al. Deep Learning and Foundation Models for Weather Prediction: A Survey, January 2025. arXiv:2501.06907 [cs.LG], http://arxiv.org/abs/2501.06907.

[34] Tél, T. et al. *Kaotikus dinamika*. Nemzeti Tankönyvkiadó Rt., Budapest, 2002.

[35] Vaswani, A. et al. Attention is All you Need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.